9 *Automating acceptance criteria for non-UI requirements 236*

9.1 Balancing UI and non-UI acceptance tests 238

9.2 When to use non-UI acceptance tests 240

9.3 Types of non-UI automated acceptance tests 243

Testing against the controller layer 243 ▪ Testing business logic directly 247 ▪ Testing the service layer 251

9.4 Defining and testing nonfunctional requirements 255

9.5 Discovering the design 257

9.6 Summary 259

10 *BDD and unit testing 260*

10.1 BDD, TDD, and unit testing 261

BDD is about writing specifications, not tests, at all levels 263 BDD builds on established TDD practices 264 ▪ BDD unit-testing tools are there to help 264

10.2 Going from acceptance criteria to implemented features 264

BDD favors an outside-in development approach 264 Start with a high-level acceptance criterion 266 Automate the acceptance criteria scenarios 267 Implement the step definitions 268 ▪ Understand the domain model 268 ▪ Write the code you'd like to have 269 Use the step definition code to specify and implement the application code 269 ▪ How did BDD help? 274

10.3 Exploring low-level requirements, discovering design, and implementing more complex functionality 274

Use step definition code to explore low-level design 275 Working with tables of examples 277 ▪ Discover new classes and services as you implement the production code 279 Implement simple classes or methods immediately 280 Use a minimal implementation 281 ▪ Use stubs and mocks to defer the implementation of more complex code 281 Expand on low-level technical specifications 283

10.4 Tools that make BDD unit testing easier 285

Practicing BDD with traditional unit-testing tools 285 Writing specifications, not tests: the RSpec family 287 Writing more expressive specifications using Spock or Spec2 291